

Project Report — Novo-Blog

Date: May 18, 2026

PROBLEM STATEMENT

- The goal of this project is to build a simple blogging platform (Novo-Blog) with a TypeScript/Node backend and a React + Vite frontend. The system must support user authentication, creating and listing posts, and viewing full post content.

INTRODUCTION

- Project structure: backend API (Express + TypeScript), frontend client (React + TypeScript + Vite).
- Key responsibilities:
 - Backend: expose REST endpoints for auth and posts, persist data (see [backend/src](#)).
 - Frontend: provide pages/components for listing, publishing, and reading posts (see [frontend/src](#)).

CODE EXPLANATION

This section explains the main parts of the code. Replace or expand each subsection with your specific implementation notes.

- Backend
 - Server entry: [backend/src/server.ts](#)
 - Database connector: [backend/src/db.ts](#)
 - Models: [backend/src/models/Post.ts](#), [backend/src/models/User.ts](#)
 - Routes: API route files live in [backend/src/routes](#)

Typical flow:

1. [server.ts](#) sets up Express, middleware, and mounts routes.
2. Routes call model/database functions in [db.ts](#) or the model files to perform CRUD.
3. Responses are JSON; errors are returned with appropriate status codes.

- Frontend
 - App entry: [frontend/src/main.tsx](#)
 - Pages: [frontend/src/pages](#)
 - Components: [frontend/src/components](#)

Typical flow:

1. [main.tsx](#) bootstraps the React app and routing.
2. Pages call API endpoints (backend) to fetch or post data.
3. Components render lists, forms, and details; hooks in [frontend/src/hooks](#) centralize common logic.

HOW TO RUN (Quick)

Backend (from **backend**):

```
npm install
npm run dev # or the start script defined in backend/package.json
```

Frontend (from **frontend**):

```
npm install
npm run dev
```

Replace the above with Yarn or pnpm commands if you use them.

OUTPUTS (SCREENSHOTS)

Include screenshots that demonstrate the application features. Place image files under **frontend/public/screenshots** (create the folder if needed) and update the paths below.

- Home / Blog list:

 Home page screenshot

- Publish page / new post form:

 Publish page screenshot

- Full post view:

 Full post screenshot

Notes on capturing screenshots:

- Run backend and frontend locally.
- Open the pages you want to document in a browser and capture screenshots (Windows: **Win+Shift+S** or a tool like Snipping Tool).
- Save PNGs into **frontend/public/screenshots** and commit them.

CONCLUSION

- Summarize results, limitations, and future work.
 - Example conclusions: basic CRUD and auth implemented; UI needs styling and pagination; add image uploads and search as future improvements.

If you want, I can:

- Insert actual code snippets from specific files into the **CODE EXPLANATION** section.
- Add real screenshots (you can upload them or point me to paths) and embed them.
- Tailor the report language (academic, business, or developer-focused).